

Kryptering og Netværkssikkerhed

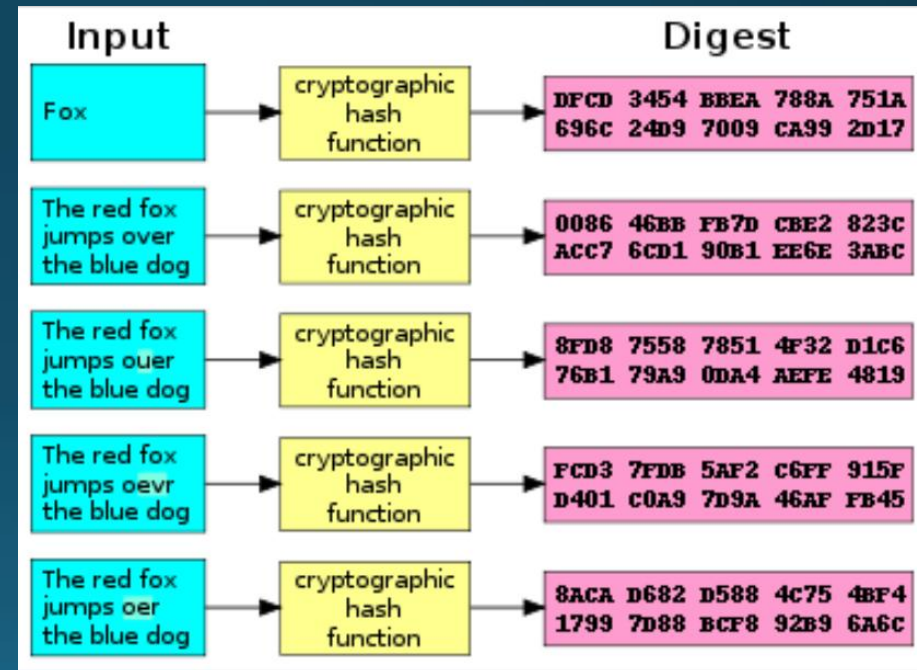
Netværk og distribueret design

Dagsorden

- Hashing
- Symmetrisk kryptering
 - Øvelse - symmetrisk kryptering og hashing
- Asymmetrisk kryptering
 - Øvelse - assymmetrisk kryptering.
- [https/tls/ssl](#)

Hashing

- Hashing er en envejs encoding
 - Input har vilkårlig størrelse
 - Output er fixed size
- Determinitisk
 - Samme input -> samme output
- Små ændringer i input
 - -> store ændringer i output



Tegn ændret også
ændres det hele

Hashing – Eksempel

- Simpel hjemmelavet hashing algoritme
 - String som input
 - ASCII værdi af alle tegn
 - Returnér totale sum

```
static string DumbHash(string input)
{
    // Add up all the character values
    int total = 0;
    foreach (char c in input)
    {
        total += (int)c;
    }

    return total.ToString();
}
```

```
> Jan
Hash: 281

> naJ
Hash: 281

> gg
Hash: 206

> a
Hash: 97

> A
Hash: 65
```

Hashing – Eksempel

- Simpel hjemmelavet hashing algoritme
 - String som input
 - ASCII værdi af alle tegn
 - Returnér totale sum
- Features:
 - Envejs ✓
 - Deterministisk ✓
 - Fixed Size ✗
 - Store ændringer i output ✗
 - Ingen overlap (collisions) ✗

```
static string DumbHash(string input)
{
    // Add up all the character values
    int total = 0;
    foreach (char c in input)
    {
        total += (int)c;
    }

    return total.ToString();
}
```

```
> Jan
Hash: 281

> naJ
Hash: 281

> gg
Hash: 206

> a
Hash: 97

> A
Hash: 65
```

Hashing – Hvorfor?

- Validering af data
 - Sammenlign 2 filer, ud fra hash-værdier
 - Nemmere end at gennemgå begge filer fra start til slut

```
C:\Users\Jan\Desktop>certutil -hashfile "abc.exe" MD5
MD5 hash of abc.exe:
b6c083eecb6a417fea17eeb28f0bd7d5
CertUtil: -hashfile command completed successfully.

C:\Users\Jan\Desktop>certutil -hashfile "def.exe" MD5
MD5 hash of def.exe:
b6c083eecb6a417fea17eeb28f0bd7d5
CertUtil: -hashfile command completed successfully.

C:\Users\Jan\Desktop>certutil -hashfile "ghi.exe" MD5
MD5 hash of ghi.exe:
504b15f2cc07e8ecff377d9084c47bbd
CertUtil: -hashfile command completed successfully.
```

Hashing – Hvorfor?

- Hashed passwords
 - Ingen grund til at gemme passwords
 - Med risiko for et læk!
 - Gem hash-værdier og sammenlign ved login!

UserId	Email	PasswordHash
0	alice@gmail.com	a9eda7a7110b0fc065ef5bd89c8b39ae
1	bob@gmail.com	557cdeb43f2276f93aa99cfda829b723
2	charlie@gmail.com	7f064cfc3fa59312c0855d3e3b71e8f5
3	delta@gmail.com	3eac36e0a264b85ee69455429f7a7c34
4	echo@gmail.com	e5f223cdc0feecb6d7c58b89d9e415b8

Hashing – Hvorfor?

- Output af hash funktioner har fast størrelse
 - MD5 → 128 bits
 - SHA1 → 160 bits
 - SHA256 → 256 bits
 - SHA512 → 512 bits
- Kryptering kræver 'keys' af bestemt størrelse (f.eks.: 16 bytes, 256 bits)
 - ... Men vi vil gerne bruge "MinKode123" (10 tegn, 20 bytes, 160 bits)
 - Køre den gennem SHA256 og få 256 bits
 - Bruge dem som 'key' og kryptere vores data

Hashing – Hvorfor?

“MyPass1234”



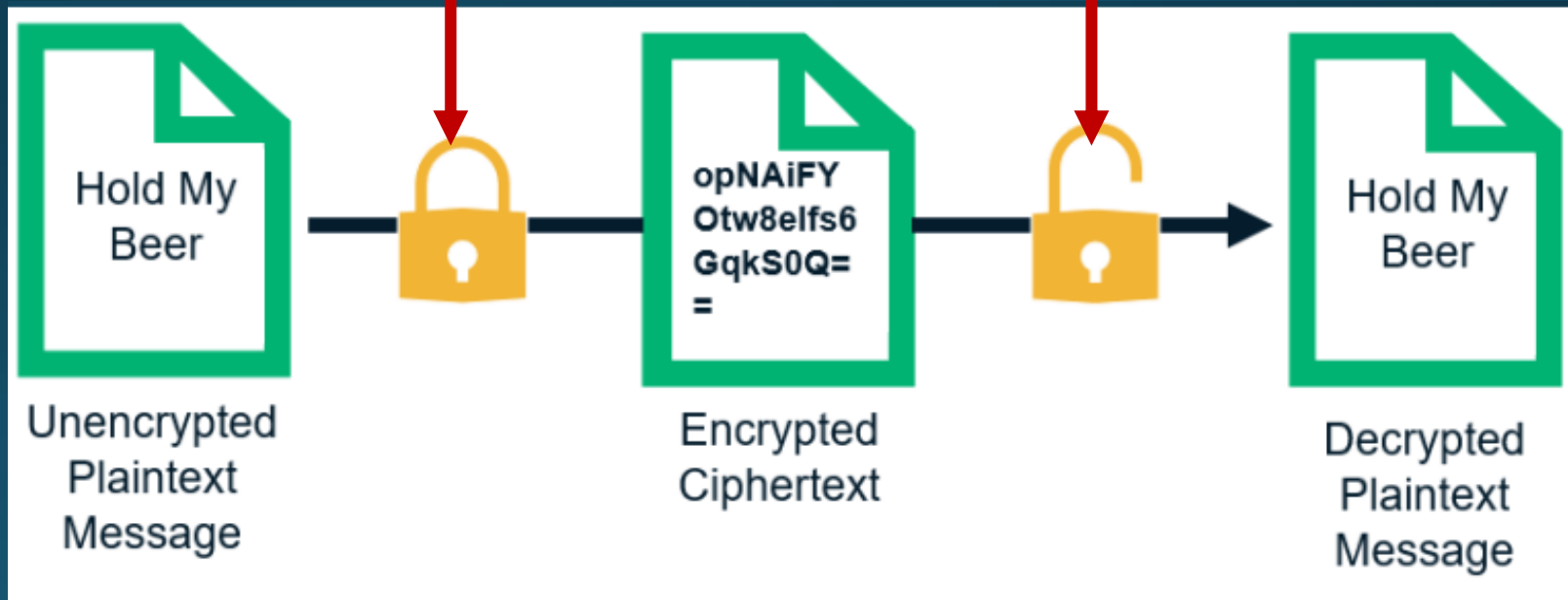
7f064cfc3
fa59312c
0855d3e
3b71e8f5

Hashing er envejs
Kryptere passwordet

“MyPass1234”



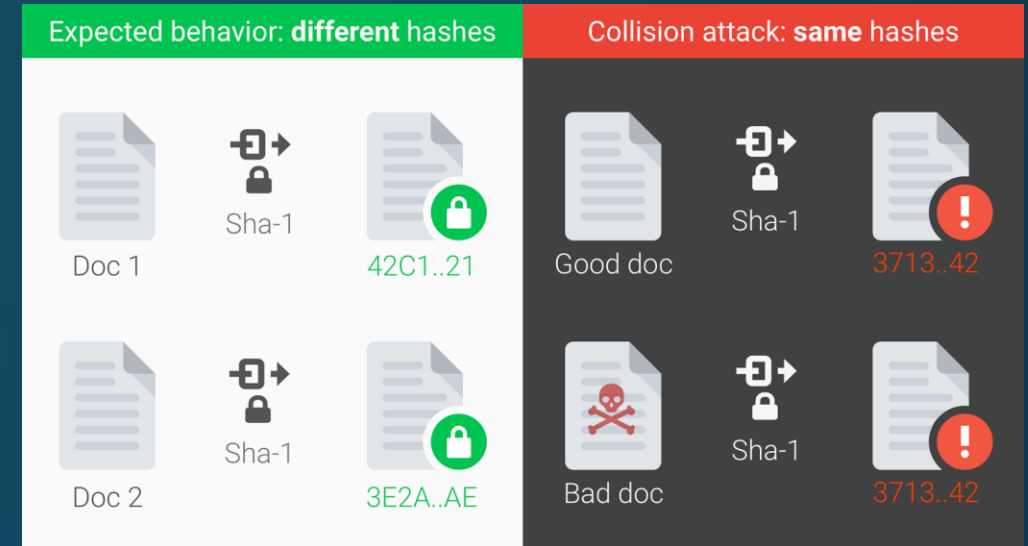
7f064cfc3
fa59312c
0855d3e
3b71e8f5



Kryptering er 2 vejs

Hashing

- Udbredte hash funktioner
 - MD5
 - SHA1
 - SHA2 (256 bits / 512 bits)
 - Argon2 (256 bits)



- Hash-collisions:

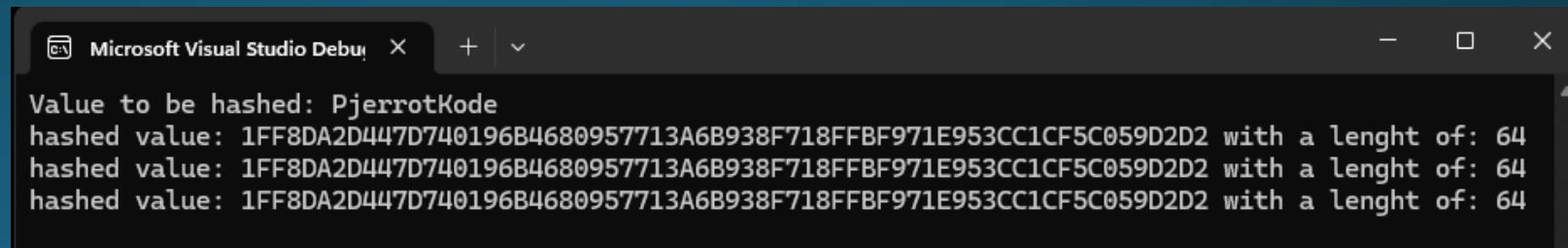
Tager højde for at et tegn har et bestemt placering

- 2 forskellige input resulterer i samme hash
- Flere bits, lavere sandsynlighed
- Kendt for MD5 og SHA1
 - Betrages som usikre (især MD5)
 - Men hurtige og kan være "gode nok"

Hashing i dotnet eksempel

- Simpel SHA256 hashing
- Computehash tager byte array som input, derfor encoding til UTF8
- BitConverter delen er blot til at gøre resultatet læseligt ☺
- Altid samme resultat
- Altid 32 bytes (256bit)
- Andre hash funktioner
 - Anden størrelse
 - F.eks Sha1 = 20 byte

```
string hashInput = "PjerrotKode";
Console.WriteLine($"Value to be hashed: {hashInput}");
for (int i = 0; i < 3; i++)
{
    SHA256 mySHA256 = SHA256.Create();
    byte[] hashVaue = mySHA256.ComputeHash(Encoding.UTF8.GetBytes(hashInput));
    string hexString = BitConverter.ToString(hashVaue).Replace("-", "");
    Console.WriteLine($"hashed value: {hexString} with a lenght of: {hexString.Length}");
}
```



```
Microsoft Visual Studio Debu  X  +  v
Value to be hashed: PjerrotKode
hashed value: 1FF8DA2D447D740196B4680957713A6B938F718FFBF971E953CC1CF5C059D2D2 with a lenght of: 64
hashed value: 1FF8DA2D447D740196B4680957713A6B938F718FFBF971E953CC1CF5C059D2D2 with a lenght of: 64
hashed value: 1FF8DA2D447D740196B4680957713A6B938F718FFBF971E953CC1CF5C059D2D2 with a lenght of: 64
```

Hashing – Salt

- Hashing er jo deterministisk
 - Samme input → Hash func. → altid samme output
 - Brugere med samme password, har samme hash gemt i databasen
 - Men vi kan tilføje "salt"

Til at gøre det mere tilfældeigt, så der fx ikke er 2 brugere med samme password har samme hash værdig

- En slags "seed" til vores hash
 - IKKE hemmelig
- Salt + input → Hash func. → andet output
 - Tilfældig salt for hver input der hashes
 - Gem salt sammen med hash-værdi

Hashing – Salt

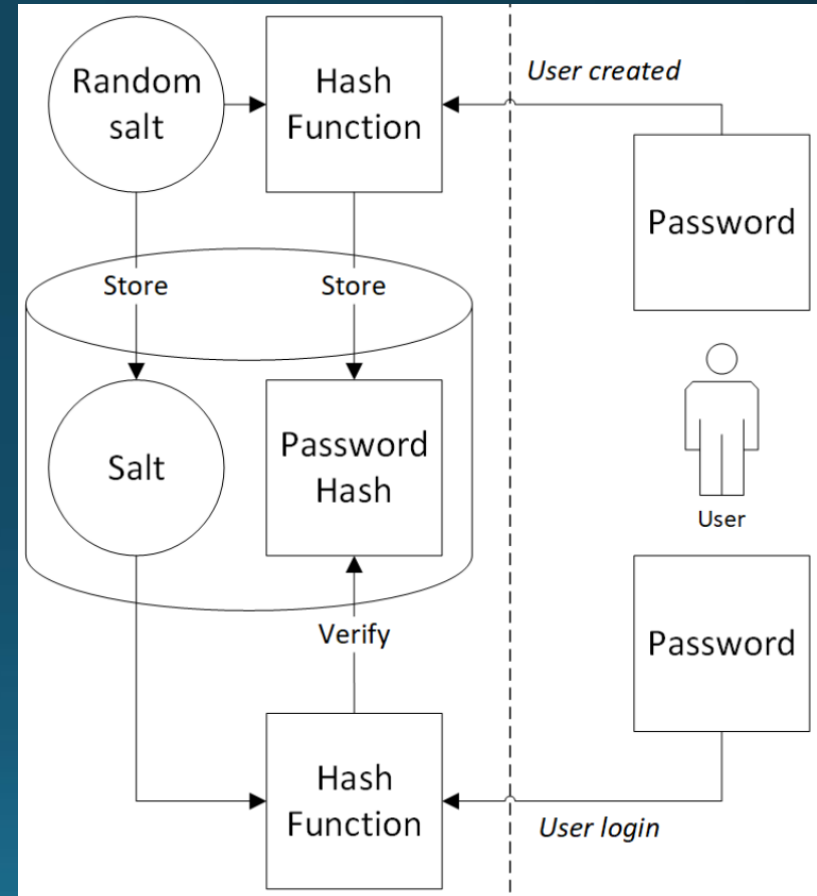
UserId	Email	PasswordHash
0	alice@gmail.com	a9eda7a7110b0fc065ef5bd89c8b39ae
1	bob@gmail.com	557cdeb43f2276f93aa99cfda829b723
2	charlie@gmail.com	7f064cfc3fa59312c0855d3e3b71e8f5
3	delta@gmail.com	3eac36e0a264b85ee69455429f7a7c34
4	echo@gmail.com	e5f223cdc0feecb6d7c58b89d9e415b8

- Hvorfor salt?
 - Vi har fået et 'leak' af en database
 - Vi kan søge efter bestemte passwords
- Vi kan søge efter hash i stedet
 - "12345" = 557cdeb43f2276f93aa99cfda829b723
- Det kan vi gøre for mange passwords
 - Kan downloade pre-computed hash-værdier (rainbow tables)

Hashing – Storing passwords

- Brug hashing + salt!
- Når vi opretter en ny bruger gør vi følgende:
 - Antag input er brugernavn og password
 - Lav ny tilfældig salt
 - Hash kombination af pass og salt
 - Gem alle brugernavn, salt og hash

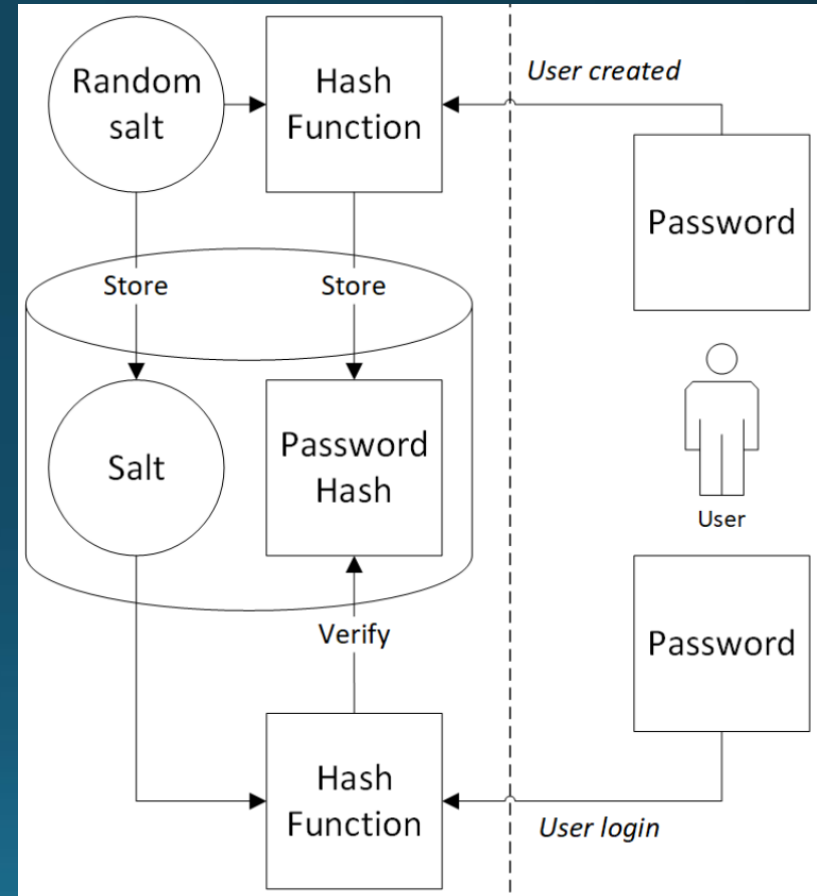
Username	hashPass	Salt
User1	Hash1	Salt1
User2	Hash2	Salt2



Hashing – Storing passwords

- Når vi skal verificere henter vi salt og kombinerer med det angivne password, og kan nu verificere mod det gemte hash
- Alle har forskellige hashes, også selvom de har samme password
- Gør derfor at hvis 1 password er cracket, kan man ikke bruge samme metode til at få andre.

Username	hashPass	Salt
User1	Hash1	Salt1
User2	Hash2	Salt2



Hashing – Eksempel

- Fuldt program til at oprette og validere brugere.
- Bruger blot et dict af string og (byte[],byte[]) i stedet for DB
 - Samme princip
 - Benytter c# tuple

```
Dictionary<string, (byte[] passHash, byte[] salt)> fakeDB;  
fakeDB = new Dictionary<string, (byte[] passHash, byte[] salt)>();  
  
AddUser("Pjerrot", "PjerrotKode");  
AddUser("Gene", "IOwnOJ");  
AddUser("Dee", "NotGonnaTakeIt");  
  
foreach (var kvp in fakeDB)  
{  
    Console.WriteLine($"{kvp.Key} has hashedPasswithSalt:" +  
        $"{BitConverter.ToString(kvp.Value.passHash).Replace("-", "")}" +  
        $" and salt: {BitConverter.ToString(kvp.Value.salt).Replace("-", "")}");  
}  
  
ValidateUser("Pjerrot", "PjerrotKode");  
ValidateUser("Dee", "NotGonnaTakeIt");
```

Microsoft Visual Studio Debug Console

```
Pjerrot has hashedPasswithSalt:3F4D73D4D55B2E226CD487C99CF884D23E664A2F3982EC1D5B1411E4F6867439 and salt: 58960A7BEEC10A10297D9E27688F0C9C  
Gene has hashedPasswithSalt:20F6844DB9502A3C43CD57945E4DDC418A22BFDE08F9F5BABD4F2F2D808A1BA5 and salt: 5FC456C69CEC0B04B859DADA6A042B18  
Dee has hashedPasswithSalt:708ECA2F12351696340BFF344EAB880C3A7CCBA52310273AF45A68C565C16861 and salt: B9D813353B35E2A94C55DEC8D9AC988E  
Pjerrot logged in with correct input and password :-)  
Dee logged in with correct input and password :-)
```


Hashing – Eksempel

- Opretter ny salt
- RNGCryptoServiceProvider
 - Genererer random byte array
- Concatinerer password og salt som byte array i værdien: PassPlusSalt
- PassPlusSalt bliver hashed med SHA256
- Bliver til sidst gemt i vores "database"

```
void AddUser(string userName, string userPass)
{
    byte[] salt = new byte[16];
    var rng = new RNGCryptoServiceProvider(); //putter ind i arr
    rng.GetBytes(salt);
    SHA256 mySHA256 = SHA256.Create();
    byte[] PassPlusSalt = Encoding.UTF8.GetBytes(userPass).Concat(salt).ToArray();
    byte[] hashedPassWithSalt = mySHA256.ComputeHash(PassPlusSalt);

    fakeDB.Add(userName, (hashedPassWithSalt, salt));
}
```

Hashing – Eksempel

- Starter med at slå op om brugeren findes
 - Vigtigt: Giver samme fejl hvis brugeren ikke findes og password ikke matcher!
 - Ellers kunne man nemt finde ud af hvilke brugere der er i systemet.
- Henter bruger info - passHash og salt
- Concatinere passwordet med det gemte salt i værdien inputPlusSalt
- inputPlusSalt hashes med SHA256
- Sammenlignes med den gemte
 - Er de ens gik et godt
 - Er ikke ens – FEJL.

```
bool ValidateUser(string userName, string password)
{
    if (fakeDB.TryGetValue(userName, out (byte[] passHash, byte[] salt) userInfo))
    {
        byte[] inputPlusSalt = Encoding.UTF8.GetBytes(password).Concat(userInfo.salt).ToArray();
        SHA256 mySHA256 = SHA256.Create();
        byte[] passPlusSaltedHashed = mySHA256.ComputeHash(inputPlusSalt);
        if (passPlusSaltedHashed.SequenceEqual(userInfo.passHash))
        {
            Console.WriteLine($"{userName} logged in with correct input and password :-)");
            return true;
        }
    }
    Console.WriteLine("incorrect user/password");
    return false;
}
```

Hashing – Eksempel

Microsoft Visual Studio Debug Console

```
Pjerrot has hashedPasswithSalt:3F4D73D4D55B2E226CD487C99CF884D23E664A2F3982EC1D5B1411E4F6867439 and salt: 58960A7BEEC10A10297D9E27688F0C9C
Gene has hashedPasswithSalt:20F6844DB9502A3C43CD57945E4DDC418A22BFDE08F9F5BABD4F2F2D808A1BA5 and salt: 5FC456C69CEC0B04B859DADA6A042B18
Dee has hashedPasswithSalt:708ECA2F12351696340BFF344EAB880C3A7CCBA52310273AF45A68C565C16861 and salt: B9D813353B35E2A94C55DEC8D9AC988E
Pjerrot logged in with correct input and password :-)
```

```
Dee logged in with correct input and password :-)
```

Hashing – rainbow tables

- Forudbergenede hash-værdier
 - Bruges som et opslagværk
 - Fylder meget! (terabytes!)
 - <https://freerainbowtables.com/>
 - Har næsten alle kombinationer

- Grunden til salt er nødvendigt
 - Øger længde og kompleksitet
 - F.eks
 - "password1" -> "password1*%W)8,-®*"
 - Gør rainbow tables nærmest ubrugelige

- Alternativ til brute-force
 - Behøver ikke udføre hashes selv
 - Men fylder mere
 - Memory/storage vs CPU kræfter



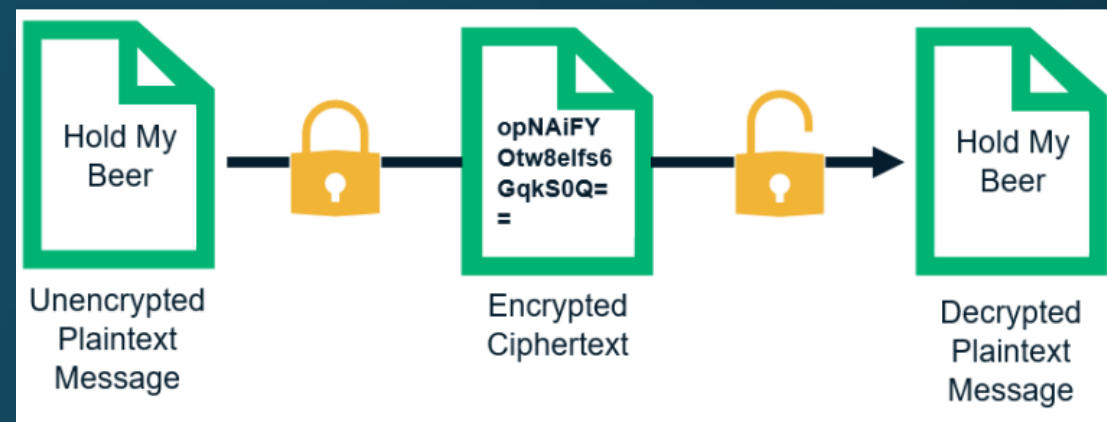
Up to 10 characters containing [abcdefghijklmnopqrstuvwxyz0123456789]

Plain text

- Alt hvad vi har arbejdet med indtil har været sendt som "plain text" / "klar tekst"
 - Egentlig som klare bytes 😊
- Dvs. at hvis blot man kender encoding formatet, kan man fange pakken og læse med
- Andre kan læse med
 - Routers mellem sender og modtager
 - Hvis i er på andres netværk
 - Eller blot gennem WIFI
- Det kan f.eks opnås med et program som Wireshark, hvor man kan opfange pakker på ens netværk

Kryptering

- Er en slags encoding
- Går to veje
 - meget ligesom serielisering og deserialisering
- Encryption/kryptering:
 - Plain text → cipher text
 - Ulæseligt
- Decryption/dekryptering:
 - Cipher text → plain text
 - Læsbar igen



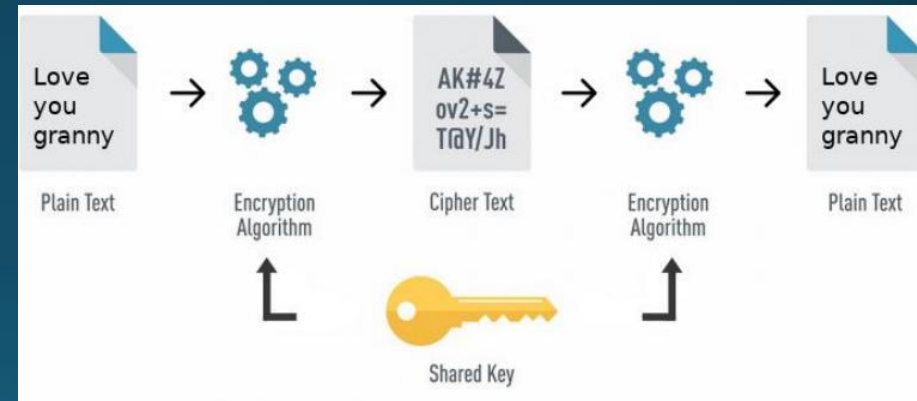
Kryptering

- Krypterings algoritmer "brydes" over tid
- Definitionen af en "brudt" algoritmet
 - "nøgler" kan findes på en måde der er hurtigere end at "brute-force"
 - Brute-force er at bare prøve en masse forskellige og håbe på man finder den rigtige.
 - F.eks.: Brute-force tager 2 mia år, men "angrebet tager "kun" 1 mia år
 - Den er principielt brudt, halvering af tiden
 - ...men er det brugbart? Ikke nu – men måske senere?
- Nogle er reelt brudt, hvor nøglen kan findes i rimelig tid
 - F.eks MD5 (tager et par timer)
 - Eller SHA1 (brudt af google)
 - <https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html>

Symmetric Encryption

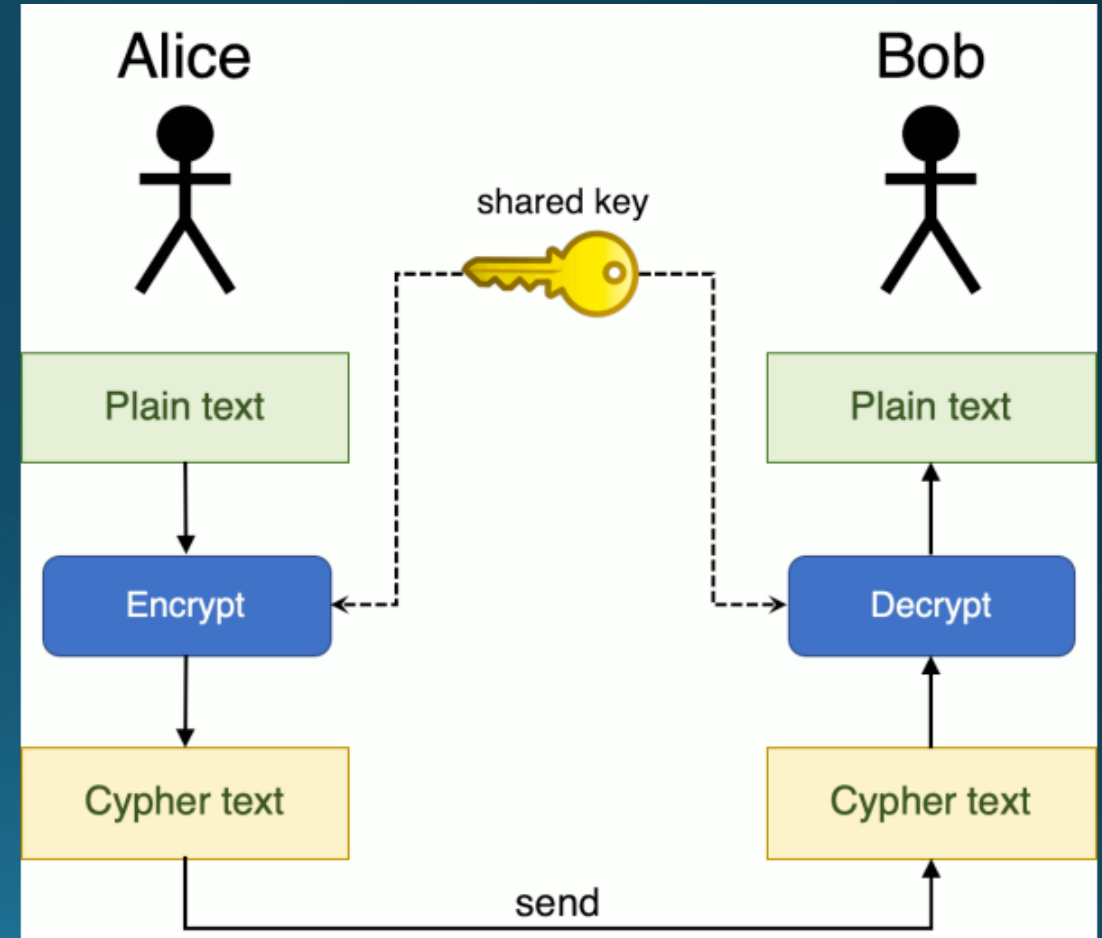
Symmetrisk Kryptering

- Symmetrisk kryptering
 - En nøgle til kryptering
 - Samme nøgle til dekryptering
 - Simpel og meget hurtig
- Oplagt til:
 - Data opbevaring (data-at-rest)
 - Store mængder af data



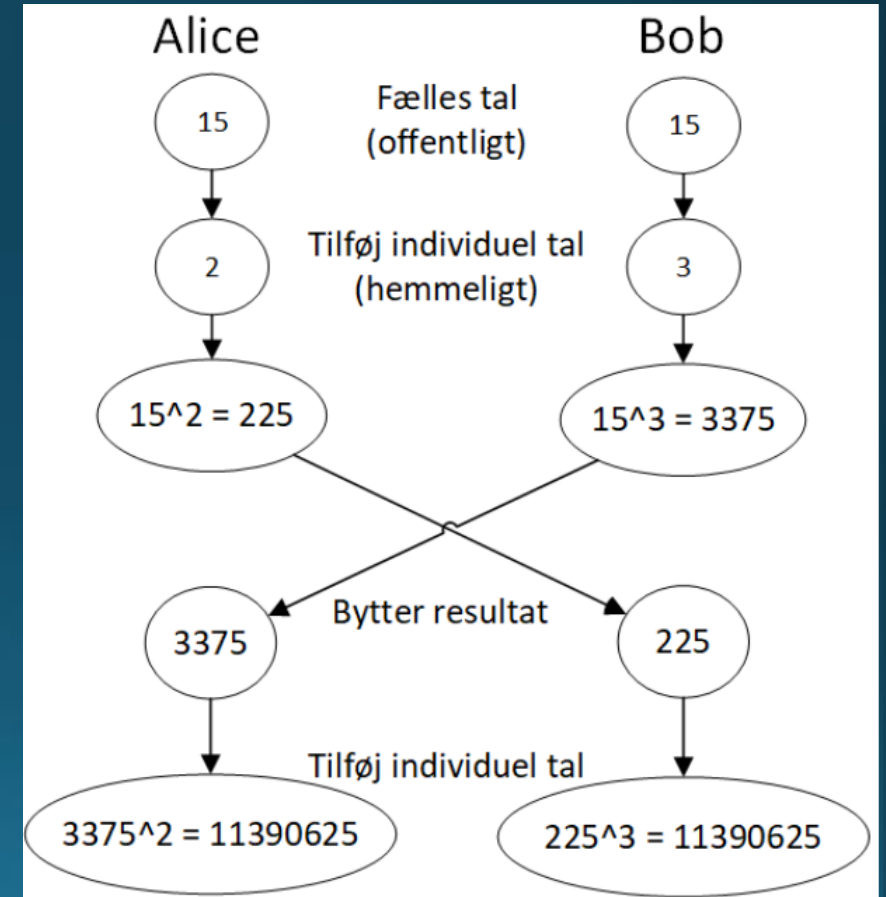
Symmetrisk Kryptering

- Over netværk?
 - Alice og bob vil kommunikere
 - Kræver de har samme nøgle
- Hvordan deler vi vores nøgle?
 - Kan vi sende den?
 - Det bliver i "plaintext"
 - ... men så kan andre opfange den

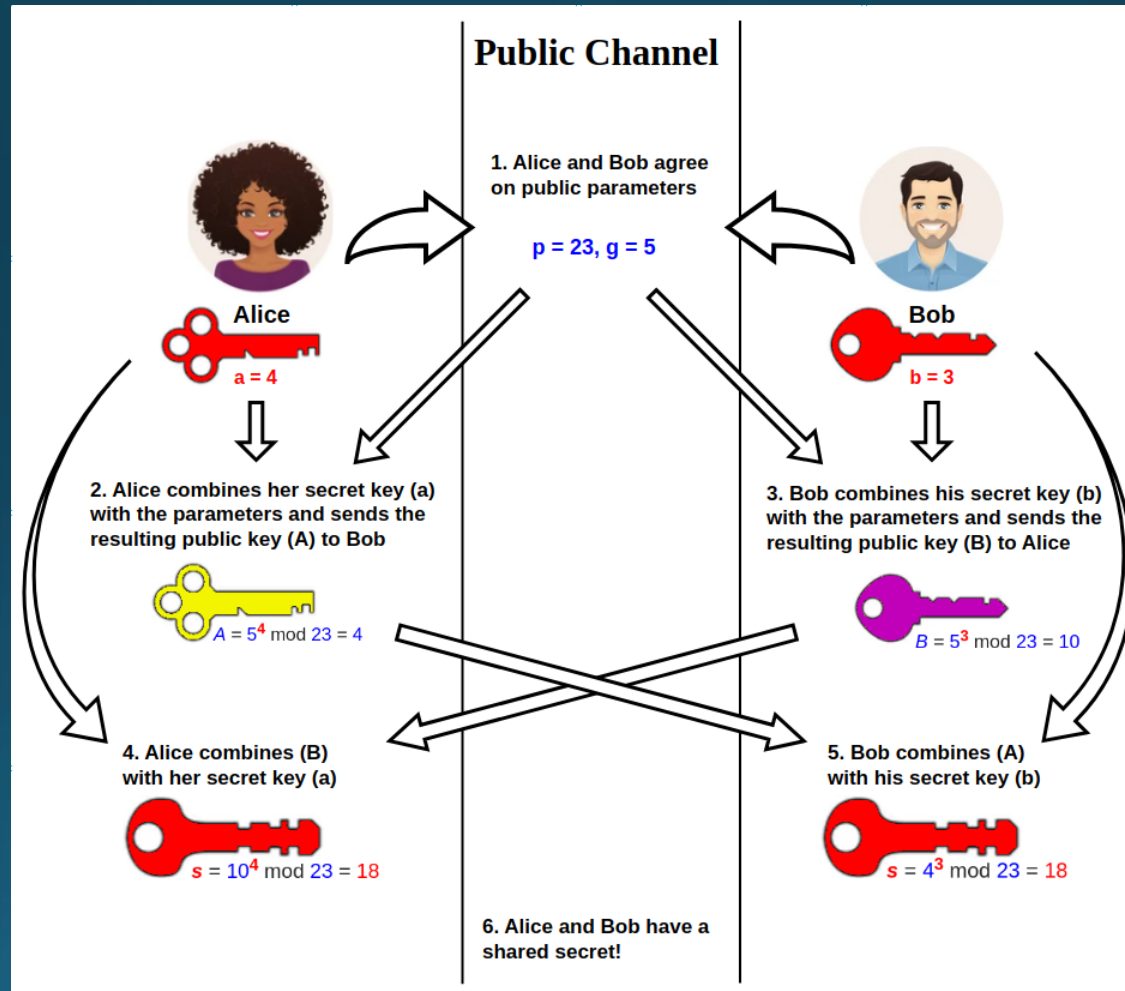


Symmetrisk Kryptering

- En 'nøgle' er bare en værdi
 - Kan Alice og Bob blive enige om en værdi/nøgle?
 - Uden at dele den direkte
- Secure Key Exchange
 - F.eks. [Diffie-Hellman](#)
- Resultatet er
 - En fælles symmetrisk nøgle

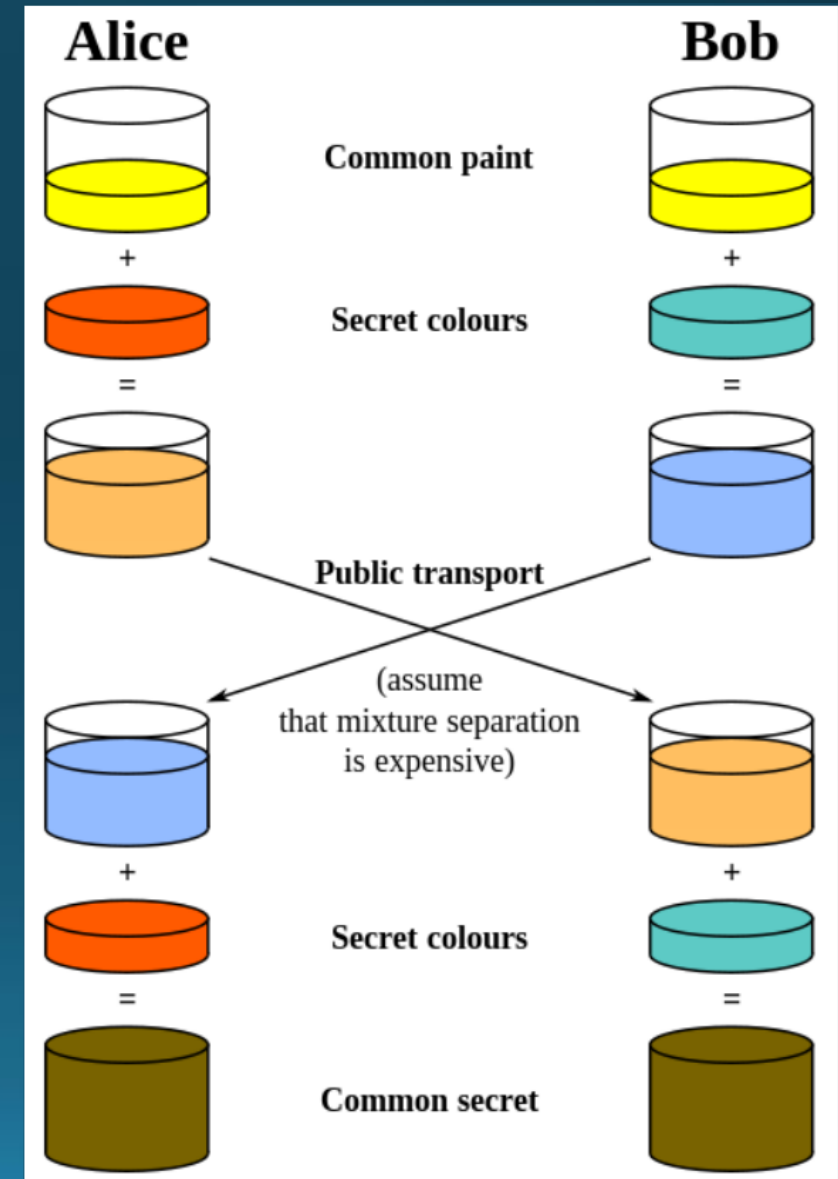


Symmetrisk Kryptering



Symmetrisk Kryptering

- Samme eksempel
 - Nu som farver



Symmetrisk kryptering algoritmer

- Udbredte symmetriske algoritmer
 - DES(data Encryption Standard),1977
 - TDES(Triple Data Encryption Standard),1995
 - AES(Advanced Encryption Standard),2001
- Key-size måles i bits (længde)
 - Generelt brugt til at beskrive hvor stærk en algoritme er
 - Større keys -> længere tid at brute force
 - ...Men også længere tid at encrypt/decrypt
 - Dvs. størrelse på key er en balance.
 - Hellere for sikker end for usikker 😊

Symmetrisk kryptering key sizes

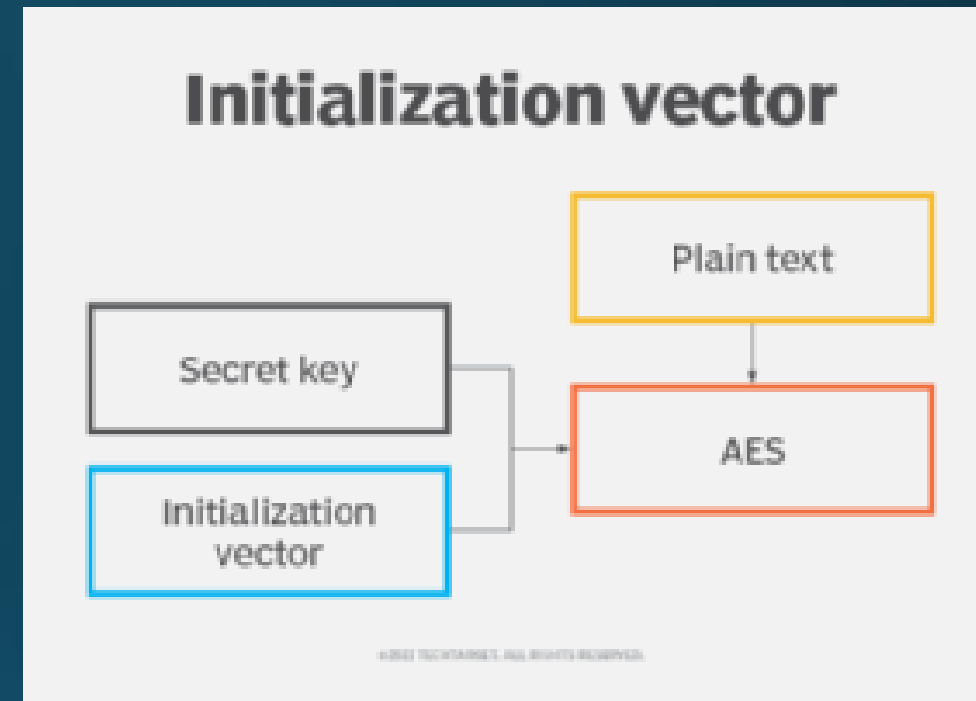
Number of Bits in Key	Odds of Cracking: 1 in	Estimated Time to Crack*
8	256	.000032 seconds
16	65,536	.008192 seconds
24	16,777,216	2.097 seconds
32	4,294,967,296	8 minutes 56.87 seconds
56	72,057,594,037,927,900	285 years 32 weeks 1 day
64	18,446,744,073,709,600,00	8,090,677,225 Years
128	3.40282E+38	5,257,322,061,209,440, 000,000 years
256	1.15792E+77	2.753,114,795,116,330,000, 000,000,000,000,000,000, 000,000,000,000 years
512	1.3408e+154	608,756,305,260,875,000,000, 000,000,000,000,000,000,000, 000,000,000,000,000,000000, 000,000,000,000,000,000,000, 000,000,000,000,000 years

[NOTE]* Estimated Time To Crack is based on a general – purpose personal computer performing eight million guesses per second.

- Vi kommer til at bruge AES, og det er her anbefalet at bruge mindst 128 bit 😊

Initialization Vector

- Som udgangspunkt
 - Samme plain text krypteres altid til samme ciphertext
- Kan tilføje en tilfældig IV får vi en anden ciphertext
 - En slags "salt" til krypterings algoritmer
 - Samme plain text + samme salt -> samme ciphertext
 - Dvs der bruges både IV og Key til dekryptering
 - IV gemmes sammen med cipherkey
 - Gemmes sammen med det krypterede data – Så minder i princippet om salt
 - Tilføjer tilfældighed i krypteringsprocessen.
 - Gør det meget meget sværere at genkende mønstre i algoritmen.
- Best practice
 - Tilfældig IV for hver gang tekst krypteres
 - IV'en bør være på 16byte (128bit) – i hvert fald i AES



AES encryption i C#

- Baseret på [denne](#)
- Stort set så nedkogt som en krypterings funktion kan være.
- Tager text, key og IV som input og returnere krypteret string som byte array.

```
byte[] Encrypt(string plainText, byte[] Key, byte[] IV)
{
    byte[] encrypted;
    // Create a new AesManaged.
    using (Aes aes = Aes.Create())
    {
        // Create encryptor
        ICryptoTransform encryptor = aes.CreateEncryptor(Key, IV);
        // Create MemoryStream
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                // Create StreamWriter and write data to a stream
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
                encrypted = ms.ToArray();
            }
        }
    }
    // Return encrypted data
    return encrypted;
}
```

using som keyword i C#

- Using statement i C#
- using er et C#-keyword
- Kan bruges til ressourser der bør lukkes/frigives efter brug (automatisk)
 - F.eks. filer, netværks-forbindelser, streams, osv...
 - Bl.a FileStream, HttpClient, MemoryStream, NetworkStream og SqlConnection
- Frigives korrekt, selvom der opstår Exceptions i kode blokken.
- Det bruges primært til at administrere objekter, der implementerer IDisposable-Interfacet.
 - Dispose()-metoden automatisk på det objekt, der er oprettet i blokken.

```
using (var resource = new SomeDisposableResource())
{
    // Brug af resource
    // Når blokken er afsluttet, kaldes Dispose() automatisk på resourcen
}
```

AES encryption i C#

- Opretter byte array til return value.
- starter med at oprette et AESManaged objekt
 - Dette objekt indeholde alle de funktioner vi har brug for at kunne kryptere med AES algoritmen

```
byte[] Encrypt(string plainText, byte[] Key, byte[] IV)
{
    byte[] encrypted;
    // Create a new AesManaged.
    using (Aes aes = Aes.Create())
    {
        // Create encryptor
        ICryptoTransform encryptor = aes.CreateEncryptor(Key, IV);
        // Create MemoryStream
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                // Create StreamWriter and write data to a stream
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
                encrypted = ms.ToArray();
            }
        }
    }
    // Return encrypted data
    return encrypted;
}
```

AES encryption i C#

- Opretter en Encryptor ved at bruge AES instansen samt key og IV
- Denne instans bruges til selve krypteringen.

```
byte[] Encrypt(string plainText, byte[] Key, byte[] IV)
{
    byte[] encrypted;
    // Create a new AesManaged.
    using (Aes aes = Aes.Create())
    {
        // Create encryptor
        ICryptoTransform encryptor = aes.CreateEncryptor(Key, IV);
        // Create MemoryStream
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                // Create StreamWriter and write data to a stream
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
                encrypted = ms.ToArray();
            }
        }
    }
    // Return encrypted data
    return encrypted;
}
```

AES encryption i C#

- opretter en ny `MemoryStream`, som fungerer som en midlertidig buffer til de krypterede data.

```
byte[] Encrypt(string plainText, byte[] Key, byte[] IV)
{
    byte[] encrypted;
    // Create a new AesManaged.
    using (Aes aes = Aes.Create())
    {
        // Create encryptor
        ICryptoTransform encryptor = aes.CreateEncryptor(Key, IV);
        // Create MemoryStream
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                // Create StreamWriter and write data to a stream
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
                encrypted = ms.ToArray();
            }
        }
    }
    // Return encrypted data
    return encrypted;
}
```

AES encryption i C#

- opretter en `CryptoStream`, der forbinder `MemoryStream` og `Encryptor`'en.
- Dette giver mulighed for at skrive krypterede data til `MemoryStream`

```
byte[] Encrypt(string plainText, byte[] Key, byte[] IV)
{
    byte[] encrypted;
    // Create a new AesManaged.
    using (Aes aes = Aes.Create())
    {
        // Create encryptor
        ICryptoTransform encryptor = aes.CreateEncryptor(Key, IV);
        // Create MemoryStream
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                // Create StreamWriter and write data to a stream
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
                encrypted = ms.ToArray();
            }
        }
    }
    // Return encrypted data
    return encrypted;
}
```

AES encryption i C#

- opretter en `StreamWriter`, der kan bruges til at skrive data til `CryptoStream`.
- I dette tilfælde skrives den ukrypterede tekst (plainText) til `CryptoStream`

```
byte[] Encrypt(string plainText, byte[] Key, byte[] IV)
{
    byte[] encrypted;
    // Create a new AesManaged.
    using (Aes aes = Aes.Create())
    {
        // Create encryptor
        ICryptoTransform encryptor = aes.CreateEncryptor(Key, IV);
        // Create MemoryStream
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                // Create StreamWriter and write data to a stream
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
            }
            encrypted = ms.ToArray();
        }
    }
    // Return encrypted data
    return encrypted;
}
```

AES encryption i C#

- konverterer `MemoryStream` til et byte array og gemmer det i `encrypted`-variablen.

```
byte[] Encrypt(string plainText, byte[] Key, byte[] IV)
{
    byte[] encrypted;
    // Create a new AesManaged.
    using (Aes aes = Aes.Create())
    {
        // Create encryptor
        ICryptoTransform encryptor = aes.CreateEncryptor(Key, IV);
        // Create MemoryStream
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                // Create StreamWriter and write data to a stream
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
                encrypted = ms.ToArray();
            }
        }
    }
    // Return encrypted data
    return encrypted;
}
```


AES encryption i C#

- returnerer det krypterede data.

```
private static string Decrypt(byte[] encryptText, byte[] Key, byte[] IV)
{
    //Create AesManaged
    using (Aes aes = Aes.Create())
    {
        ICryptoTransform decryptor = aes.CreateDecryptor(Key, IV);

        //Create MemoryStream
        using (MemoryStream ms = new MemoryStream(encryptText))
        {
            using (CryptoStream cs = new CryptoStream(ms, decryptor, CryptoStreamMode.Read))
            {
                //Create StreamReader
                using (StreamReader sr = new StreamReader(cs))
                {
                    return sr.ReadToEnd();
                }
            }
        }
    }
}
```

```
byte[] Encrypt(string plainText, byte[] Key, byte[] IV)
{
    byte[] encrypted;
    // Create a new AesManaged.
    using (Aes aes = Aes.Create())
    {
        // Create encryptor
        ICryptoTransform encryptor = aes.CreateEncryptor(Key, IV);
        // Create MemoryStream
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                // Create StreamWriter and write data to a stream
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
                encrypted = ms.ToArray();
            }
        }
    }
    // Return encrypted data
    return encrypted;
}
```

Øvelse – symmetrisk kryptering

- Se følgende eksempel for hjælp:
 - Denne er hvad gennemgangen var baseret på
- Lav en console app der kan kryptere en besked
 - Samme applikation skal kunne dekryptere en besked
- Sørg for at det er muligt at specificere sin egen kode ("key")
 - Dvs. hash brugerens input string til en key 😊
- Brug unik IV til hver kryptering
- Gør det muligt at kunne kryptere en besked, stoppe programmet og starte det op igen, hvor det så skal være muligt at læse beskeden igen.
 - Alternativt lad to forskellige programmer kunne kryptere og dekryptere en string.

Øvelse – symmetrisk kryptering hints

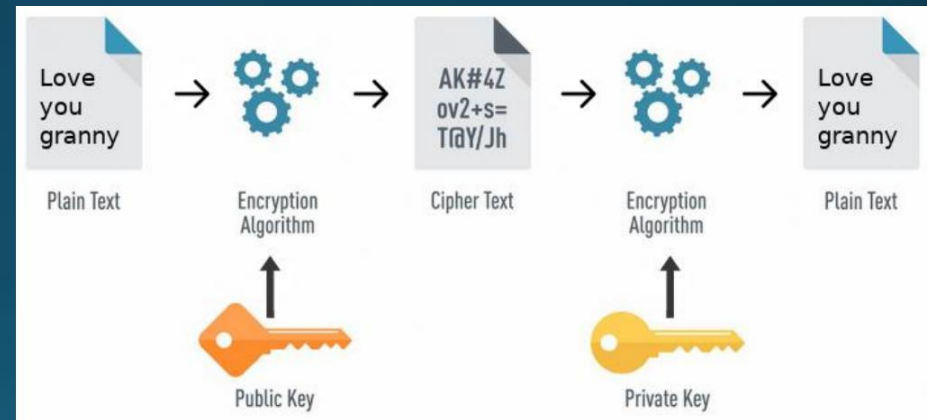
- Key og IV kunne f.eks laves sådan her:

```
byte[] key = SHA256.HashData(Encoding.UTF8.GetBytes("hello"));  
byte[] iv = new byte[16];  
var rng = new RNGCryptoServiceProvider();  
rng.GetBytes(iv);
```

Asymmetric Encryption

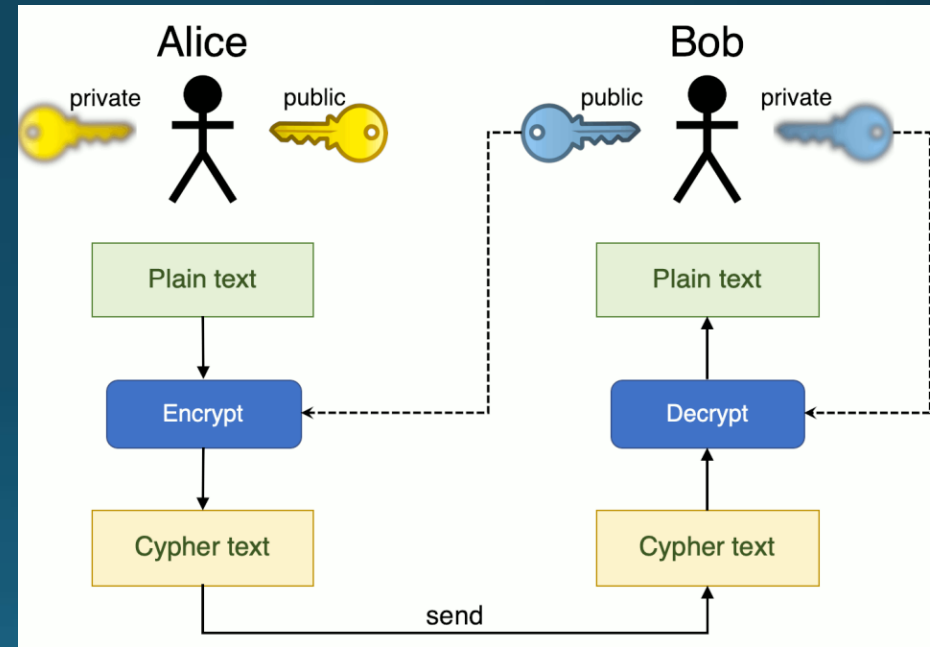
Asymmetrisk Kryptering

- Et nøgle-par
 - Den ene nøgle krypterer
 - Den anden nøgle dekrypterer
- Public key
 - er offentlig
 - Bruges til at kryptere
 - Enhver kan få denne og kryptere data
- Private key
 - Er hemmelig/fortrolig
 - Bruges til at dekryptere data
 - Kun ejeren af private key kan dekryptere



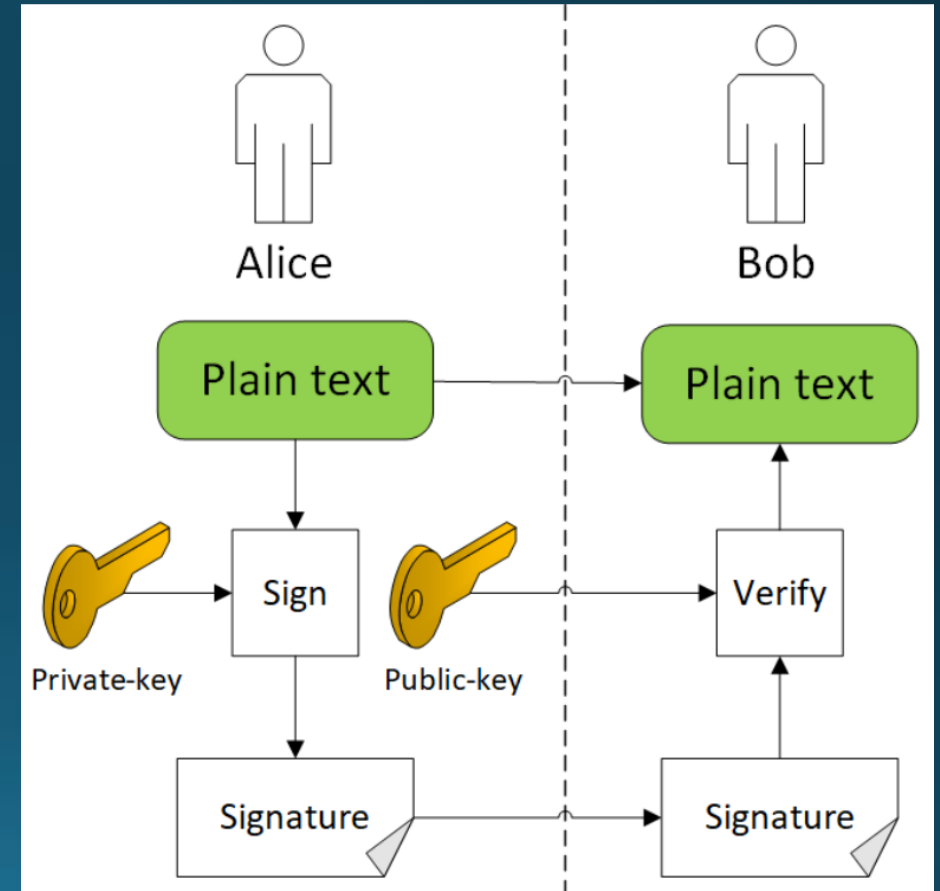
Asymmetrisk Kryptering

- Over netværk
 - Bobs Public-key er tilgængelig
 - Alice krypterer med public Bob's key
 - Kun Bob kan dekryptere
 - Kun bob har private key
- Private key forbliver privat
 - Ingen udveksling
- Alle kan snakke hemmeligt til bob.



Asymmetrisk Kryptering

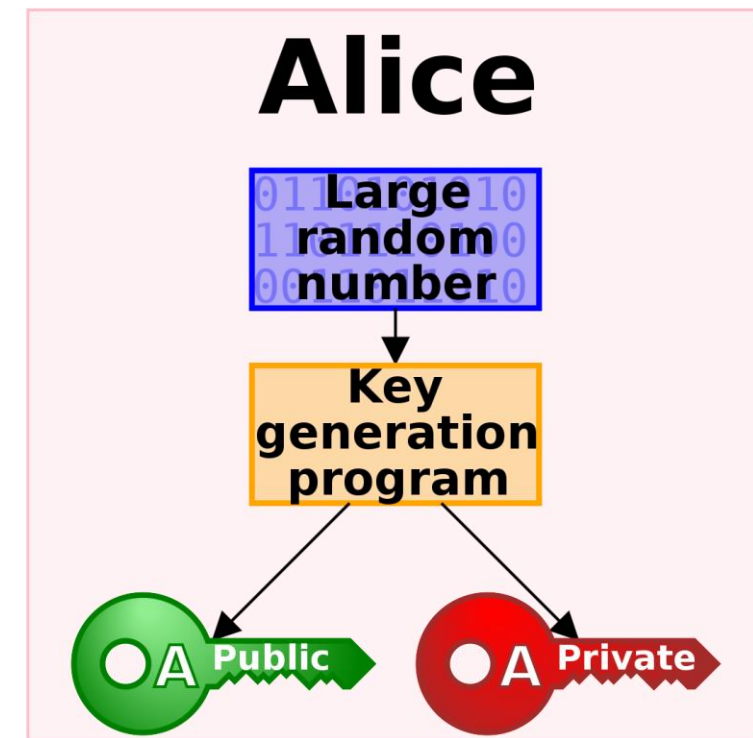
- Alternativt brug – At signere beskeder
 - For at validere afsender
 - Omvendt nøglebrug
 - Private → Krypterer
 - Public → Dekrypterer
1. Alice hasher og "signerer" beskeden
 - Krypterer med private key
 2. Sender besked og signatur
 3. Bob dekrypterer signatur med public key
 4. Bob sammenligner med plain text
 - Er de ens, er Alice afsenderen



Asymmetrisk Kryptering

- Et nøglepar bestående af offentlig nøgle og privat nøgle genereres
- Matematisk forbundet, men privat nøgle kan ikke udledes fra den offentlige nøgle
- Algoritmer til asymmetrisk kryptering
 - RSA (Rivest-Shamir-Adleman)
 - Mest udbredte – Den vi bruger i øvelserne
 - DSA (Digital Signature Algorithm)
 - ECC (Elliptic Curve Cryptography)

Faktor	RSA	DSA	ECC
Nøglegenerering	Tidkrævende	Hurtig	Hurtig
Nøglestørrelse	Større nøgler er nødvendige	Mindre nøgler er nødvendige	Mindre nøgler er nødvendige
Hastighed	Langsom ved kryptering/afkryptering af store data	Hurtig ved signering/verificering	Hurtig ved signering/verificering
Signaturens størrelse	Større	Mindre	Mindre
Kryptografisk sikkerhed	Meget sikker	Sikker	Meget sikker
Brug i SSL/TLS	Ja	Nej	Ja
Kompatibilitet	Bredt understøttet	Begrænset understøttelse	Bredt understøttet
Anvendelsesområde	Generel anvendelse	Primært til digitale signaturer	Generel anvendelse



RSA i .net

- Simplere rent kode mæssigt end symmetrisk kryptering.
- Mere avanceret bag scenerne
 - Men det er gemt væk 😊

```
using (RSA rsa = RSA.Create())
{
    // Kryptér og dekryptér en besked
    string message = "Dette er en hemmelig besked!";
    string publicKey = rsa.ToXmlString(false);
    string privateKey = rsa.ToXmlString(true);
    byte[] encryptedData = EncryptData(message, publicKey);
    string decryptedMessage = DecryptData(encryptedData, privateKey);

    Console.WriteLine("Original besked: " + message);
    Console.WriteLine("Krypteret data: " + Convert.ToBase64String(encryptedData));
    Console.WriteLine("Dekrypteret besked: " + decryptedMessage);
}

byte[] EncryptData(string data, string publicKey)
{
    byte[] byteData = Encoding.UTF8.GetBytes(data);

    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(publicKey);
        return rsa.Encrypt(byteData, RSAEncryptionPadding.Pkcs1);
    }
}

string DecryptData(byte[] encryptedData, string privateKey)
{
    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(privateKey);
        byte[] decryptedData = rsa.Decrypt(encryptedData, RSAEncryptionPadding.Pkcs1);
        return Encoding.UTF8.GetString(decryptedData);
    }
}
```

RSA i .net

- Oprettelse af RSA objektet
 - Dette skal bruges til at oprette vores key pair
 - Vi benytter den statiske "Create"
 - Dette sikre vi får den anbefalede implementering afhængig af platform vi er på.
 - Der findes bla. Også RSACng
 - Denne er specifik til windows...
 - Ved at bruge create er vi ikke bundet til en specifikt implementation af RSA

```
using (RSA rsa = RSA.Create())
{
    // Kryptér og dekryptér en besked
    string message = "Dette er en hemmelig besked!";
    string publicKey = rsa.ToXmlString(false);
    string privateKey = rsa.ToXmlString(true);
    byte[] encryptedData = EncryptData(message, publicKey);
    string decryptedMessage = DecryptData(encryptedData, privateKey);

    Console.WriteLine("Original besked: " + message);
    Console.WriteLine("Krypteret data: " + Convert.ToBase64String(encryptedData));
    Console.WriteLine("Dekrypteret besked: " + decryptedMessage);
}

byte[] EncryptData(string data, string publicKey)
{
    byte[] byteData = Encoding.UTF8.GetBytes(data);

    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(publicKey);
        return rsa.Encrypt(byteData, RSAEncryptionPadding.Pkcs1);
    }
}

string DecryptData(byte[] encryptedData, string privateKey)
{
    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(privateKey);
        byte[] decryptedData = rsa.Decrypt(encryptedData, RSAEncryptionPadding.Pkcs1);
        return Encoding.UTF8.GetString(decryptedData);
    }
}
```

RSA i .net

1. Oprettelse af string der skal krypteres
2. Eksportering af af private og public key
 - Når vi opretter et RSA objekt opretter vi også et nøgle sæt
3. Kalder funktion til at kryptere data
 - Params:
 - String message
 - private key
4. Kalder funktion til at dekryptere data
 - Params:
 - Krypteret data
 - Public key

```
using (RSA rsa = RSA.Create())
{
    // Kryptér og dekryptér en besked
    string message = "Dette er en hemmelig besked!";
    string publicKey = rsa.ToXmlString(false);
    string privateKey = rsa.ToXmlString(true);
    byte[] encryptedData = EncryptData(message, publicKey);
    string decryptedMessage = DecryptData(encryptedData, privateKey);

    Console.WriteLine("Original besked: " + message);
    Console.WriteLine("Krypteret data: " + Convert.ToBase64String(encryptedData));
    Console.WriteLine("Dekrypteret besked: " + decryptedMessage);
}

byte[] EncryptData(string data, string publicKey)
{
    byte[] byteData = Encoding.UTF8.GetBytes(data);

    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(publicKey);
        return rsa.Encrypt(byteData, RSAEncryptionPadding.Pkcs1);
    }
}

string DecryptData(byte[] encryptedData, string privateKey)
{
    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(privateKey);
        byte[] decryptedData = rsa.Decrypt(encryptedData, RSAEncryptionPadding.Pkcs1);
        return Encoding.UTF8.GetString(decryptedData);
    }
}
```

RSA i .net

1. Konvertering af string til byte array
2. Oprettelse af nyt RSA object
 1. Ikke genbrugt grundet at dette kunne være i et andet scope.
3. Importering af vores public key, så denne kan benyttes til at udføre selve krypteringen

```
using (RSA rsa = RSA.Create())
{
    // Kryptér og dekryptér en besked
    string message = "Dette er en hemmelig besked!";
    string publicKey = rsa.ToXmlString(false);
    string privateKey = rsa.ToXmlString(true);
    byte[] encryptedData = EncryptData(message, publicKey);
    string decryptedMessage = DecryptData(encryptedData, privateKey);

    Console.WriteLine("Original besked: " + message);
    Console.WriteLine("Krypteret data: " + Convert.ToBase64String(encryptedData));
    Console.WriteLine("Dekrypteret besked: " + decryptedMessage);
}

byte[] EncryptData(string data, string publicKey)
{
    byte[] byteData = Encoding.UTF8.GetBytes(data);

    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(publicKey);
        return rsa.Encrypt(byteData, RSAEncryptionPadding.Pkcs1);
    }
}

string DecryptData(byte[] encryptedData, string privateKey)
{
    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(privateKey);
        byte[] decryptedData = rsa.Decrypt(encryptedData, RSAEncryptionPadding.Pkcs1);
        return Encoding.UTF8.GetString(decryptedData);
    }
}
```

RSA i .net

- Vi bruger RSA-objektet til at kryptere byte-arrayet, ved hjælp af Encrypt.
- Her bruger vi "RSAEncryptionPadding.Pkcs1", som andet parameter
- Dette er Padding algoritmen
 - Data bliver paddet i overensstemmelse med PKCS #1-v1.5-standarderne.
 - Data får korrekt længde, og noget randomisering, som undgår nogle angreb
- Samme padding skal bruges ved decrypt.

```
using (RSA rsa = RSA.Create())
{
    // Kryptér og dekryptér en besked
    string message = "Dette er en hemmelig besked!";
    string publicKey = rsa.ToXmlString(false);
    string privateKey = rsa.ToXmlString(true);
    byte[] encryptedData = EncryptData(message, publicKey);
    string decryptedMessage = DecryptData(encryptedData, privateKey);

    Console.WriteLine("Original besked: " + message);
    Console.WriteLine("Krypteret data: " + Convert.ToBase64String(encryptedData));
    Console.WriteLine("Dekrypteret besked: " + decryptedMessage);
}

byte[] EncryptData(string data, string publicKey)
{
    byte[] byteData = Encoding.UTF8.GetBytes(data);

    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(publicKey);
        return rsa.Encrypt(byteData, RSAEncryptionPadding.Pkcs1);
    }
}

string DecryptData(byte[] encryptedData, string privateKey)
{
    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(privateKey);
        byte[] decryptedData = rsa.Decrypt(encryptedData, RSAEncryptionPadding.Pkcs1);
        return Encoding.UTF8.GetString(decryptedData);
    }
}
```

RSA i .net

- Køres programmet fåes følgende:

```
using (RSA rsa = RSA.Create())
{
    // Kryptér og dekryptér en besked
    string message = "Dette er en hemmelig besked!";
    string publicKey = rsa.ToXmlString(false);
    string privateKey = rsa.ToXmlString(true);
    byte[] encryptedData = EncryptData(message, publicKey);
    string decryptedMessage = DecryptData(encryptedData, privateKey);

    Console.WriteLine("Original besked: " + message);
    Console.WriteLine("Krypteret data: " + Convert.ToBase64String(encryptedData));
    Console.WriteLine("Dekrypteret besked: " + decryptedMessage);
}

byte[] EncryptData(string data, string publicKey)
{
    byte[] byteData = Encoding.UTF8.GetBytes(data);

    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(publicKey);
        return rsa.Encrypt(byteData, RSAEncryptionPadding.Pkcs1);
    }
}

string DecryptData(byte[] encryptedData, string privateKey)
{
    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(privateKey);
        byte[] decryptedData = rsa.Decrypt(encryptedData, RSAEncryptionPadding.Pkcs1);
        return Encoding.UTF8.GetString(decryptedData);
    }
}
```

Microsoft Visual Studio Debu

+ v

— □ ×

Original besked: Dette er en hemmelig besked!

Krypteret data: Nyc07AWAOxG4JmFc3TvduRuovArXbczj3CWl0PAFKKgWSK+c3tGvm8Ctj8nbvEcYi83MZpM1+khey5qgvUf70+PxMhF2D7HNw7WvAlTX
hnh6ycuGd3SFH9nE31BL8AZ4mtEU5yGnhfIKl/BFnx0p9vbpgtPc0mbnVxNBsaKJLUBIBG57k/wu9TSPHT1kAcEVtN/EwZEKAF0BB09/Ny0JAj7usDsx2MpG
eVI2RSPFo5/AR0xbXkqjERlTbt4sHlC/wtr4vyeHz+OADyfozspQbOMrTSWmv/yFtvviqdvZqDuREFsQoNbd24o/mGVDpIFCXAh52gP7V05oVu1GosxCw==

Dekrypteret besked: Dette er en hemmelig besked!

RSA i .net

- Vi får et forskelligt resultat af encrypt hver gang.
- Altid samme resultat ved decrypt!

```
Microsoft Visual Studio Debug Console
Original besked: Dette er en hemmelig besked!
Krypteret data: UE6GoXAerY...
Dekrypteret besked: Dette er en hemmelig besked!
Krypteret data: NOcRzEuTim...
Dekrypteret besked: Dette er en hemmelig besked!
Krypteret data: D0ebKCLzKO...
Dekrypteret besked: Dette er en hemmelig besked!
Krypteret data: ZNOJAPU1TL...
Dekrypteret besked: Dette er en hemmelig besked!
Krypteret data: C7w5JIinTd...
Dekrypteret besked: Dette er en hemmelig besked!
```

```
// Generer en nøglepar
using System.Security.Cryptography;
using System.Text;

using (RSA rsa = RSA.Create())
{
    // Kryptér og dekryptér en besked
    string message = "Dette er en hemmelig besked!";
    string publicKey = rsa.ToXmlString(false);
    string privateKey = rsa.ToXmlString(true);

    Console.WriteLine("Original besked: " + message);
    for (int i = 0; i < 5; i++)
    {
        byte[] encryptedData = EncryptData(message, privateKey);
        string decryptedMessage = DecryptData(encryptedData, publicKey);
        Console.WriteLine("Krypteret data: " +
            Convert.ToBase64String(encryptedData).Substring(0, 10) + "...");
        Console.WriteLine("Dekrypteret besked: " + decryptedMessage);
    }
}

byte[] EncryptData(string data, string publicKey)
{
    byte[] byteData = Encoding.UTF8.GetBytes(data);

    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(publicKey);
        return rsa.Encrypt(byteData, RSAEncryptionPadding.Pkcs1);
    }
}

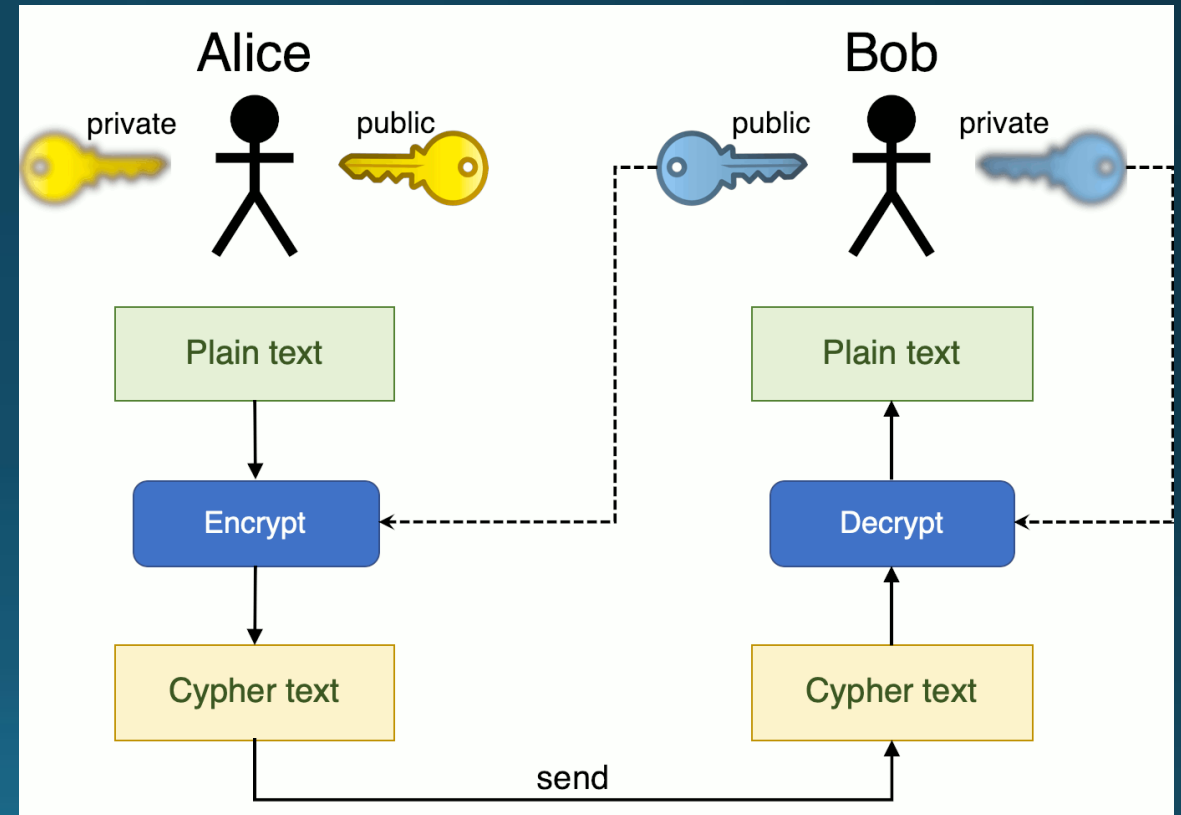
string DecryptData(byte[] encryptedData, string privateKey)
{
    using (RSA rsa = RSA.Create())
    {
        rsa.FromXmlString(privateKey);
        byte[] decryptedData = rsa.Decrypt(encryptedData, RSAEncryptionPadding.Pkcs1);
        return Encoding.UTF8.GetString(decryptedData);
    }
}
```

Symmetrisk vs Asymmetrisk

- Samme nøgle til både kryptering og dekryptering.
 - Er hurtig og effektiv til kryptering og dekryptering af store datamængder.
 - Kræver sikker distribution af den delte nøgle til alle parter.
 - Velegnet til beskyttelse af lagrede data og hurtig kryptering af kommunikation på et sikkert netværk.
 - Anvendes ofte i scenarier, hvor hastighed og effektivitet er vigtige, og hvor der allerede er etableret et sikkert netværk.
- Separate nøgler til kryptering og dekryptering
 - Giver mulighed for sikker kommunikation uden behov for sikker distribution af den delte nøgle.
 - Velegnet til sikker udveksling af nøgler og digital signatur.
 - Er mere beregningstung og langsommere end symmetrisk kryptering.
 - Anvendes ofte i scenarier, hvor der er behov for stærk sikkerhed og identitetsbekræftelse, f.eks. SSL/TLS-protokollen.
- Asymmetrisk bruges normal til at sikre kommunikation og udveksling af nøgler
 - Symmetrisk kan bruges efterfølgende til selve dataen!

Øvelse – Asymmetrisk kryptering

- Lav jeres egen key-pair
 - Public / private keys
- Krypter og dekrypter en besked
 - Med jeres egne nøgler
- Byt nu public key med sidemanden
 - Evt. gennem netværket...?
 - Krypter en besked med dén key
- Giv den krypterede besked tilbage
- Dekrypter den besked du fik
- Kort demo...



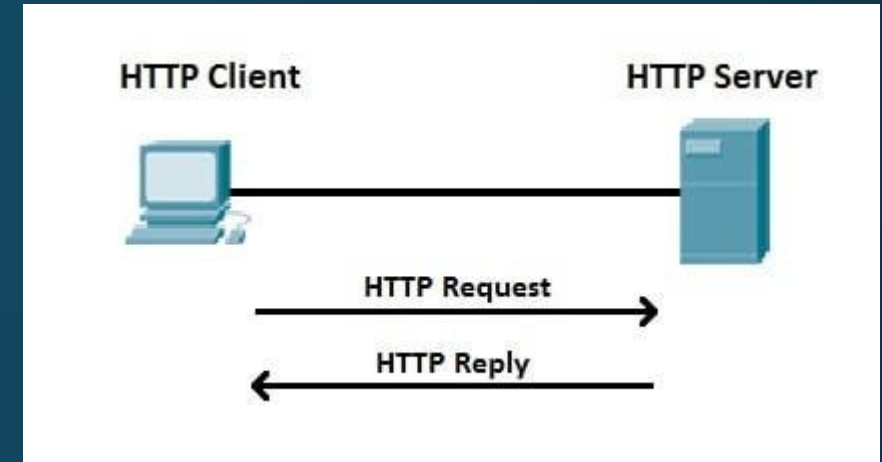
HTTPS, SSL og TLS

HTTPS, SSL og TLS

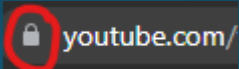
- For at forstå sikker internet kommunikation skal vi have disse termer på plads
- https – hyper text transfer protocol secure
 - En sikker (krypteret) form for HTTP
- SSL - Secure sockets layer
 - Sørger for kryptering af vores socket forbindelser
- TLS – Transfer layer security
 - Essentielt en ny form for SSL. SSL bliver ikke brugt så meget længere
- Har alle sammen samme formål, at kryptere de bekedet der bliver sendt over internettet

HTTP

- Bruger port 80
- Laver request til server får svar (reply) tilbage
 - Request: <http://google.com>
 - Reply: html document med google.com
- får svar tilbage i plain text.
 - Dvs hvis man "sniffer" pakken der bliver sendt kan man læse alt data der bliver sendt og modtaget.
 - Ikke optimalt til data så som passwords
- Foregår på lag 7 (application) laget af OSI modellen
- Letvægts protokol, men usikker.



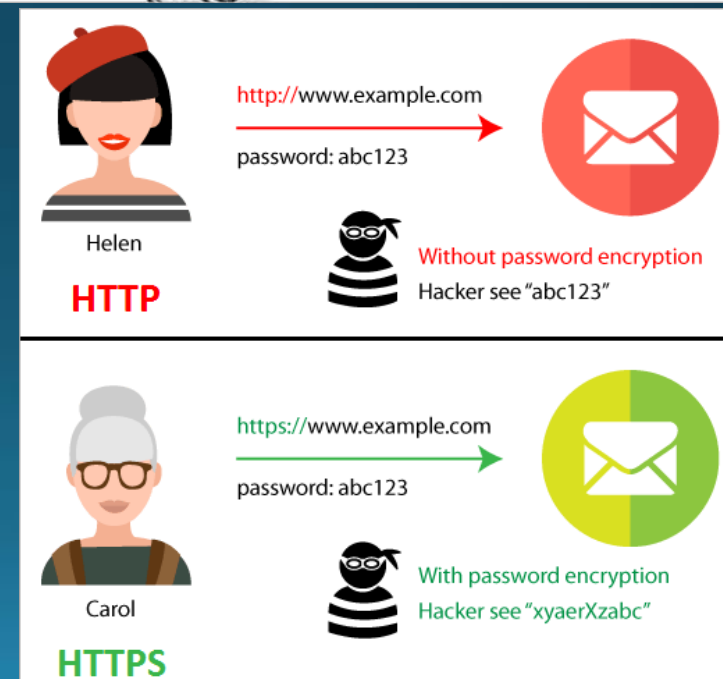
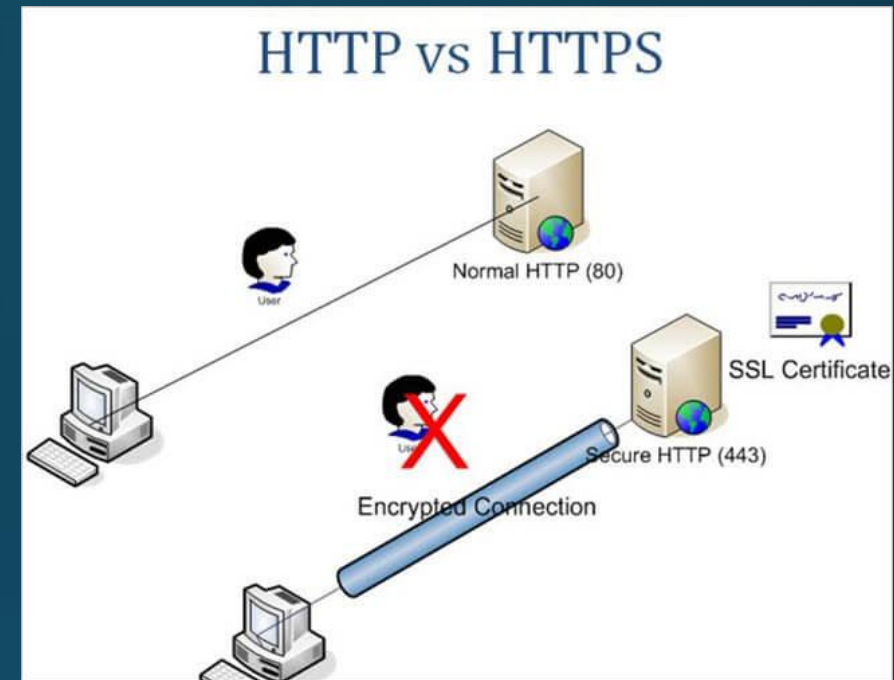
HTTPS

- Bruger port 443
- Har et ekstra lag af sikkerhed, s'et i HTTPS
 - S'et står for security
 - Egentlig HTTP over SSL/TLS
 - OBS: SSL er den ældre version af TLS, men begge udtryk bruges ofte om hinanden.
- I stedet for at sende plaintext er dataen krypteret og dekrypteret
 - I browsere indikeres dette med hængelås eller lign. ikon 
- Bliver krypteret når dataen bliver sendt af klienten, og bliver dekrypteret af serveren
 - Samme gælder når serveren sender data, krypteret af serveren, dekrypteret af klienten
- Derfor er der ikke meget ved at "sniffe" dataen, da det er volapyk
 - Er stadig muligt at se hvem der kommunikerer(ip'er), hvilken port(e) der bliver brugt osv...



HTTP vs HTTPS

HTTP	HTTPS
Bruger port 80	Bruger port 443
Plaintext ergo den sendte data kan opfanges, og er derfor muligt at lave "man in the middle attacks"	Krypteret og sikker
Bruger ingen Certifikat	Bruger SSL/TLS certifikat
Hurtig, grundet simplicitet	Langsommere end ren HTTP



TLS handshake

- Så hvad foregår der egentlig når der skal etableres en sikker og krypteret forbindelse?
- TLS handshake som er bygget ind i protokollen, eller givet direkte af sikkerheds library.
- Bruger asymmetrisk kryptering til handshake.
- Bruger symmetrisk kryptering til data overførelse.
- Foregår bag scenerne
 - Er forbindelse opsat til at bruge TLS sker det automatisk
 - Kræver et certifikat givet fra en CA (certificate authority)



CA (Certificate authority)

- Betroet enhed eller organisation, der udsteder digitale certifikater til at bekræfte identiteten på internettet.
 - Bruges altså til at svare på om et domæne er hvem det udgiver sig for at være.
 - Kræver derfor at man har ejer et domæne for at bruge.
- CAs etablerer tillid og sikkerhed i SSL/TLS-kommunikation og digitale certifikater
- 2 typer CA:
 - Offentlige
 - Offentligt anerkendte CA'er, der udsteder certifikater til at validere identiteten af websteder.
 - Private
 - Opererer internt i organisationer og udsteder certifikater til interne formål.
 - F.eks til development